MOSART TRAINING

Digital Design and RTL Project

Project 8

Programmable Memory Controller

RTL Design, Functional Verification and Synthesis

Submitted by	Shaun Praneeth Raj
Programme	B.Tech, Electronics and Communication Engineering
Institution	Anurag University, Hyderabad
Target Device	Xilinx Artix-7 (xc7a35tcpg236-1)
Date	26 September 2026

Abstract

This report presents the design, verification and synthesis of a programmable synchronous memory controller for a small on-chip memory. The controller gives another digital block a simple request interface (`read_enable`, `write_enable`, `address`, `write_data`) and returns `read_data` together with `busy` and `ready` status. A six-state Moore machine (IDLE, READ, WRITE, WAIT, DONE, ERROR) sequences each access, drives the memory strobes and inserts a run-time programmable number of wait states. A small configuration register bank allows the controller to be enabled or disabled, write protected, limited to a programmable address window, and interrogated through a sticky status register. Invalid requests (simultaneous read and write, out-of-range address, write under protection, request while busy) are rejected without touching memory and are recorded in the status register.

The design is written in synthesizable Verilog-2005 and is fully synchronous with a single clock and a synchronous active-high reset. A self-checking testbench with a behavioural reference model runs ten test groups, including a 500-access constrained-random regression, and completes 3356 checks with zero errors. Synthesis for the Xilinx 7-series fabric maps the design to 59 LUTs, one distributed RAM primitive (RAM32M) and 44 flip-flops, with a worst register-to-register path of six logic levels.

Contents

Abstract **1**

1 Problem Statement **5**

2 Requirements and Specifications **5**

2.1 System Requirements 5

2.2 Functional Specification 5

2.3 Input and Output Signals 6

2.4 Register Map 6

2.5 Clock and Reset Architecture 7

2.6 Interface Definition 7

3 Architecture and Block Diagram **7**

4 Module Hierarchy **8**

5 FSM Diagram **9**

6 Datapath Diagram **10**

7 RTL Source Code **10**

7.1 mem_ctrl_top.v 10

7.2 mem_ctrl_fsm.v 11

7.3 cfg_regs.v 13

7.4 sync_mem.v 15

8 Testbench **15**

9 Simulation Results **21**

9.1 Simulation Log 21

9.2 Simulation Waveforms 21

9.3 Important Signal Explanations 27

9.4 Verification Summary 27

10 Synthesis Results **27**

11 Timing Analysis **28**

12 Resource Utilization **29**

13 Design Challenges **29**

14 Possible Improvements **30**

15 Conclusion **30**

List of Figures

1 System block diagram of the programmable memory controller 7
2 Module hierarchy 8
3 Controller state transition diagram (Moore machine) 9
4 Datapath: request latch, memory array, read register, validation logic and wait counter 10
5 Back-to-back writes with zero wait states 22
6 Back-to-back reads with zero wait states 22
7 Write and read of address 0xF with three programmed wait states 23
8 Simultaneous read and write request rejected 24
9 Address window limited to eight words 25
10 Request arriving while the controller is busy 26
11 Synchronous reset applied in the middle of a read 26

List of Tables

1 Functional requirements and how the design meets them 5
2 Top-level port list of `mem_ctrl_top` 6
3 Configuration and status register map 6
4 Source files and responsibilities 8
5 State encoding and Moore outputs 9
6 Verification plan 16
7 Signals shown in the waveforms 27
8 Evidence that every specified function has been verified 27
9 Yosys `synth_xilinx` cell report for `mem_ctrl_top` 28

10 Deepest logic paths in the mapped netlist 28
11 Resource utilization on xc7a35tcpg236-1 (from the Yosys 7-series mapping) . . . 29
12 Flip-flop budget by function 29

1 Problem Statement

Design and implement a programmable synchronous memory controller for a small on-chip memory. The controller shall provide a simple request interface that allows another digital block to perform memory read and write operations. It shall accept read and write requests, generate the memory control signals, return read data, report BUSY and READY status, and handle invalid or simultaneous read and write requests appropriately. The memory may be implemented as a register array, and a suitable controller may be built around an FSM containing IDLE, READ, WRITE, WAIT and DONE states.

The project follows the complete digital design flow: architecture definition, RTL design, functional verification by simulation, and synthesis with analysis of LUT usage, flip-flop usage and timing.

2 Requirements and Specifications

2.1 System Requirements

Table 1: Functional requirements and how the design meets them

ID	Requirement	Implementation
R1	Accept read requests	`read_enable` sampled in IDLE; FSM enters READ
R2	Accept write requests	`write_enable` sampled in IDLE; FSM enters WRITE
R3	Generate memory control signals	`mem_re`, `mem_we`, `mem_addr`, `mem_wdata` decoded from FSM state and request latch
R4	Provide read data	Registered memory output drives `read_data`, valid when `ready` is high and held until the next read
R5	BUSY / READY status	`busy` high in every non-IDLE state; `ready` one-cycle completion pulse
R6	Handle simultaneous read and write	Request rejected in the ERROR state, memory untouched, STATUS.COLL set
R7	Handle invalid requests	Out-of-range address, write under protection and request while busy are rejected and flagged
R8	Programmability	Enable, write protect, 0 to 7 wait states and address limit set at run time through the configuration port
R9	Register-array memory	16 × 8-bit array with synchronous write and registered read

2.2 Functional Specification

- Memory organisation: 16 words of 8 bits (parameters `ADDR_W` = 4, `DATA_W` = 8).
- One transaction in flight at a time. A request is a single-cycle pulse on `read_enable` or `write_enable` issued while `busy` is low.
- Access latency: with N programmed wait states, `ready` is asserted $N + 2$ clock cycles after

the edge that samples the request. The controller returns to IDLE one cycle later, so the maximum rate is one access every $N + 3$ cycles.

- Request validation order in IDLE: collision (both enables high) first, then address range (`address` $\geq$ LIMIT), then write protection. The first failing check sends the FSM to ERROR.
- A rejected request completes in two cycles with `ready` and `error` high together for one cycle.
- While EN = 0 the controller ignores requests entirely: `busy` and `ready` stay low.

2.3 Input and Output Signals

Table 2: Top-level port list of `mem_ctrl_top`

Signal	Dir	Width	Description
`clk`	in	1	System clock, 100 MHz target
`reset`	in	1	Synchronous reset, active high
`read_enable`	in	1	Read request (one-cycle pulse)
`write_enable`	in	1	Write request (one-cycle pulse)
`address`	in	4	Word address of the request
`write_data`	in	8	Data to be written
`read_data`	out	8	Data returned by a read, valid with `ready`
`ready`	out	1	Transaction finished (one-cycle pulse)
`busy`	out	1	Transaction in progress, new requests not accepted
`error`	out	1	Request rejected (pulse coincident with `ready`)
`cfg_we`	in	1	Configuration register write strobe
`cfg_addr`	in	2	Configuration register address
`cfg_wdata`	in	8	Configuration write data
`cfg_rdata`	out	8	Configuration read-back (combinational)

2.4 Register Map

Table 3: Configuration and status register map

Addr	Name	Bits	Access	Reset	Function
0x0	CTRL	[0]	RW	1	EN: controller enable
		[1]	RW	0	WP: write protect, writes rejected
0x1	WAIT	[2:0]	RW	0	Wait states inserted per access (0 to 7)
0x2	LIMIT	[4:0]	RW	16	Number of valid words; values above 16 saturate to 16
0x3	STATUS	[0]	W1C	0	COLL: simultaneous read and write request
		[1]	W1C	0	RANGE: address $\geq$ LIMIT
		[2]	W1C	0	WPV: write attempted while WP = 1
		[3]	W1C	0	OVR: request dropped while busy
		[4]	RO	0	BUSY: live copy of `busy`

W1C bits are cleared by writing 1 to them. If an error event and a clear occur in the same cycle, the event wins so that no error is lost.

2.5 Clock and Reset Architecture

The whole design sits in one clock domain driven by `clk`; there are no derived or gated clocks and therefore no clock domain crossings. Reset is synchronous and active high, the style recommended for Xilinx 7-series flip-flops (FDRE/FDSE), because it keeps the reset in the data path and avoids asynchronous recovery and removal checks. Reset returns the FSM to IDLE and the configuration registers to their defaults. The memory array itself is deliberately not reset: this allows it to map onto distributed RAM, and a controller reset does not destroy stored data (verified in test T9).

2.6 Interface Definition

The host protocol is a simple valid/ready handshake with one outstanding request:

1. The host waits for `busy` = 0, then drives `address`, `write_data` and one enable for a single clock cycle.
2. The controller latches the request, raises `busy` on the next cycle and ignores further requests (flagging them as OVR).
3. When `ready` pulses, the transaction is complete. For a read, `read_data` is valid in that cycle and remains stable until the next read. If `error` is also high the request was rejected and the STATUS register gives the reason.

3 Architecture and Block Diagram

The controller is partitioned into a control path (the FSM), a configuration path (register bank) and a storage path (memory array), as shown in Figure 1. The host sees only the top-level ports; all memory strobes are internal.

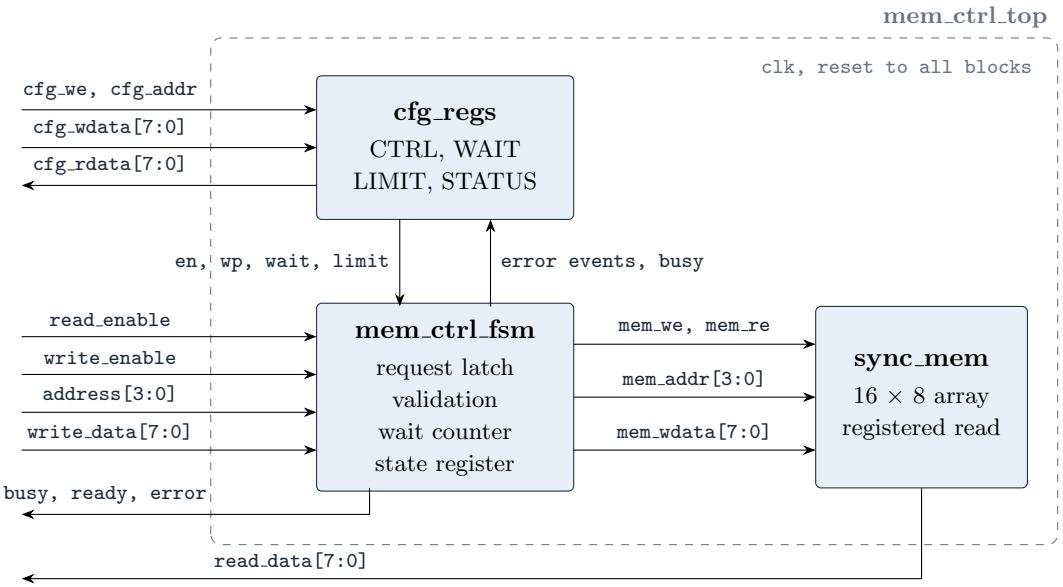


Figure 1: System block diagram of the programmable memory controller

Datapath. Request address and write data are captured into `addr_q` and `wdata_q` when a request is accepted in IDLE, so the host may change its inputs immediately. The memory array is written from these latched values and its registered output forms `read_data` directly. A 3-bit down-counter (`wcnt`) measures the programmed wait states.

Control path. The FSM is a Moore machine: every output (`busy`, `ready`, `error`, `mem_we`, `mem_re`) is a decode of the state register only, so the outputs are glitch-free and have no combinational path from the request inputs. Only the next-state logic in IDLE looks at the request and configuration inputs.

Configuration path. The register bank holds the programmable parameters and collects one-cycle error events from the FSM into sticky STATUS bits.

4 Module Hierarchy

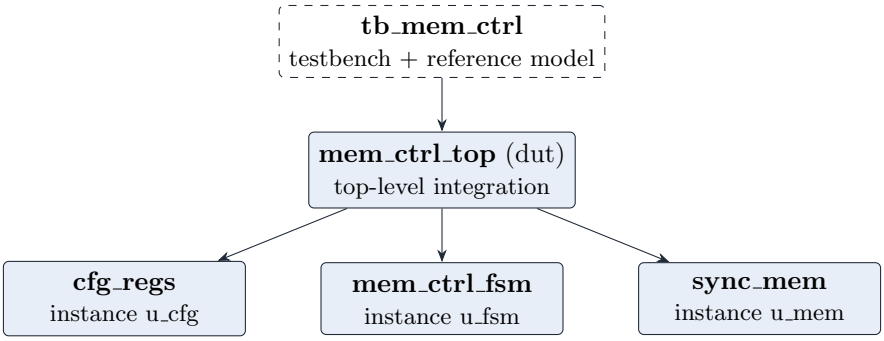


Figure 2: Module hierarchy

Table 4: Source files and responsibilities

File	Module	Responsibility
`rtl/mem_ctrl_top.v`	`mem_ctrl_top`	Structural top; wires the three sub-blocks
`rtl/mem_ctrl_fsm.v`	`mem_ctrl_fsm`	State machine, request latch, validation, wait counter, error events
`rtl/cfg_regs.v`	`cfg_regs`	CTRL, WAIT, LIMIT, STATUS registers and read-back mux
`rtl/sync_mem.v`	`sync_mem`	16 × 8 register-array memory, synchronous write, registered read
`tb/tb_mem_ctrl.v`	`tb_mem_ctrl`	Self-checking testbench with reference model

5 FSM Diagram

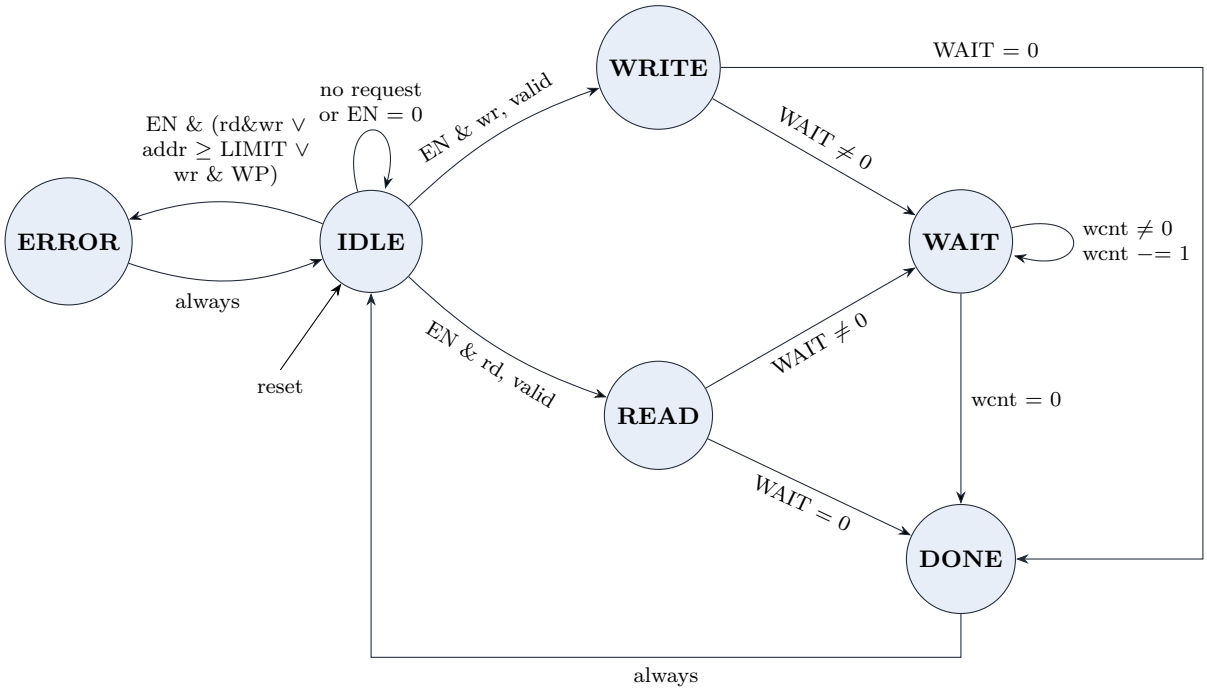


Figure 3: Controller state transition diagram (Moore machine)

Table 5: State encoding and Moore outputs

State	Code	busy	ready	error	mem_we	mem_re	Action
IDLE	000	0	0	0	0	0	Wait for request, latch and validate it
READ	001	1	0	0	0	1	Memory read strobe; data registered at end of cycle
WRITE	010	1	0	0	1	0	Memory write strobe; array updated at end of cycle
WAIT	011	1	0	0	0	0	Count down the programmed wait states
DONE	100	1	1	0	0	0	Signal completion; `read_data` valid
ERROR	101	1	1	1	0	0	Signal rejection; memory not accessed

Binary encoding was kept deliberately: with six states it needs three flip-flops, and Yosys reported that re-encoding would bring no benefit for this output decode.

6 Datapath Diagram

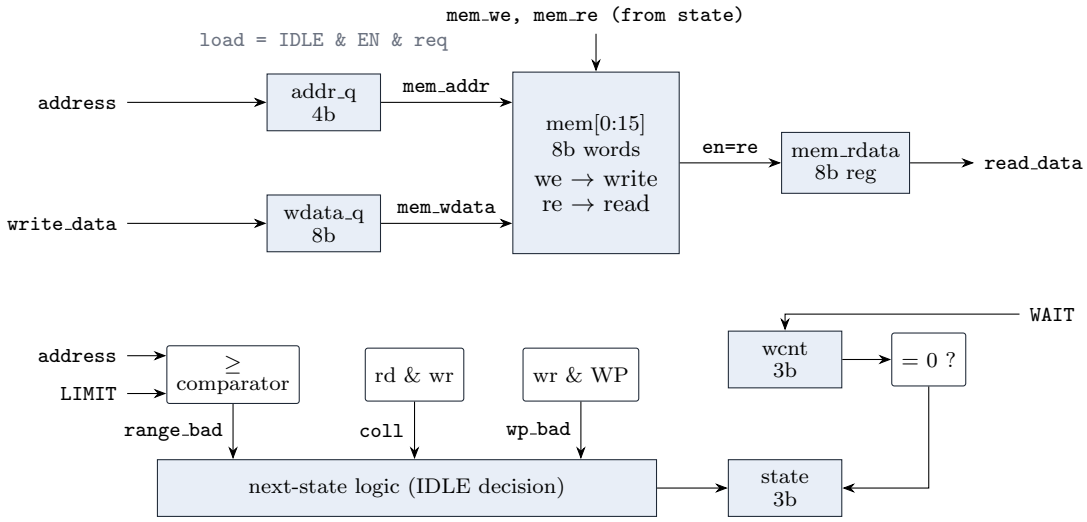


Figure 4: Datapath: request latch, memory array, read register, validation logic and wait counter

The longest combinational path in the design runs from `address` and the LIMIT register through the $\geq$ comparator and the IDLE next-state decision into the state register (Section 11). All memory-side signals leave registers directly (`addr_q`, `wdata_q`, state decode), so the memory write and read paths are short.

7 RTL Source Code

All modules are written in synthesizable Verilog-2005. Sequential logic uses non-blocking assignments in clocked `always` blocks; combinational logic uses `always @(*)` with a default assignment at the top so that no latches are inferred. The `initial` block in `sync_mem` only sets the power-up contents of the array, which Vivado honours as an FPGA initial value.

7.1 mem_ctrl_top.v

```
//==============================================================================
// Module      : mem_ctrl_top
// Project     : MOSART Project 8 - Programmable Memory Controller
// Author      : Shaun Praneeth Raj
// Description : Top level. Connects the configuration register bank, the
//               controller FSM and the 16 x 8 synchronous register-array
//               memory. All logic is in a single clock domain (clk) with a
//               synchronous active-high reset.
//==============================================================================
`timescale 1ns / 1ps
module mem_ctrl_top #(
    parameter DATA_W = 8,
    parameter ADDR_W = 4
)(
    input  wire             clk,
    input  wire             reset,
    // host request interface
```

```
    input  wire              read_enable,
    input  wire              write_enable,
    input  wire [ADDR_W-1:0] address,
    input  wire [DATA_W-1:0] write_data,
    output wire [DATA_W-1:0] read_data,
    output wire              ready,
    output wire              busy,
    output wire              error,
    // configuration interface
    input  wire              cfg_we,
    input  wire [1:0]        cfg_addr,
    input  wire [7:0]        cfg_wdata,
    output wire [7:0]        cfg_rdata
);
    wire              en, wp;
    wire [2:0]        wait_states;
    wire [ADDR_W:0]   addr_limit;
    wire              mem_we, mem_re;
    wire [ADDR_W-1:0] mem_addr;
    wire [DATA_W-1:0] mem_wdata;
    wire              ev_coll, ev_range, ev_wpv, ev_ovr;

    cfg_regs #(.ADDR_W(ADDR_W)) u_cfg (
        .clk(clk), .reset(reset),
        .cfg_we(cfg_we), .cfg_addr(cfg_addr),
        .cfg_wdata(cfg_wdata), .cfg_rdata(cfg_rdata),
        .en(en), .wp(wp), .wait_states(wait_states), .addr_limit(addr_limit),
        .ev_coll(ev_coll), .ev_range(ev_range), .ev_wpv(ev_wpv),
        .ev_ovr(ev_ovr), .busy(busy)
    );

    mem_ctrl_fsm #(.DATA_W(DATA_W), .ADDR_W(ADDR_W)) u_fsm (
        .clk(clk), .reset(reset),
        .read_enable(read_enable), .write_enable(write_enable),
        .address(address), .write_data(write_data),
        .ready(ready), .busy(busy), .error(error),
        .en(en), .wp(wp), .wait_states(wait_states), .addr_limit(addr_limit),
        .mem_we(mem_we), .mem_re(mem_re),
        .mem_addr(mem_addr), .mem_wdata(mem_wdata),
        .ev_coll(ev_coll), .ev_range(ev_range), .ev_wpv(ev_wpv), .ev_ovr(ev_ovr)
    );

    sync_mem #(.DATA_W(DATA_W), .ADDR_W(ADDR_W)) u_mem (
        .clk(clk),
        .mem_we(mem_we), .mem_re(mem_re),
        .mem_addr(mem_addr), .mem_wdata(mem_wdata),
        .mem_rdata(read_data)
    );
endmodule
```

7.2 mem_ctrl_fsm.v

```
//============================================================================
// Module      : mem_ctrl_fsm
// Project     : MOSART Project 8 - Programmable Memory Controller
// Author      : Shaun Praneeth Raj
// Description : Control path. Accepts one request at a time from the host,
//               validates it, drives the memory strobes, inserts the
//               programmed number of wait states and signals completion.
//
//   States : IDLE -> READ  -> [WAIT] -> DONE -> IDLE
```

```
//            IDLE -> WRITE -> [WAIT] -> DONE -> IDLE
//            IDLE -> ERROR -> IDLE              (rejected request)
//
//   busy  = 1 in every state except IDLE
//   ready = 1 for one cycle in DONE or ERROR (transaction finished)
//   error = 1 for one cycle in ERROR (request rejected, memory untouched)
//==============================================================================
`timescale 1ns / 1ps
module mem_ctrl_fsm #(
    parameter DATA_W = 8,
    parameter ADDR_W = 4
)(
    input  wire              clk,
    input  wire              reset,
    // host request interface
    input  wire              read_enable,
    input  wire              write_enable,
    input  wire [ADDR_W-1:0] address,
    input  wire [DATA_W-1:0] write_data,
    output wire              ready,
    output wire              busy,
    output wire              error,
    // programmed configuration
    input  wire              en,
    input  wire              wp,
    input  wire [2:0]        wait_states,
    input  wire [ADDR_W:0]   addr_limit,
    // memory control
    output wire              mem_we,
    output wire              mem_re,
    output wire [ADDR_W-1:0] mem_addr,
    output wire [DATA_W-1:0] mem_wdata,
    // error events to the status register
    output reg               ev_coll,
    output reg               ev_range,
    output reg               ev_wpv,
    output reg               ev_ovr
);
    localparam [2:0] S_IDLE  = 3'd0,
                     S_READ  = 3'd1,
                     S_WRITE = 3'd2,
                     S_WAIT  = 3'd3,
                     S_DONE  = 3'd4,
                     S_ERROR = 3'd5;

    reg [2:0]        state, next_state;
    reg [ADDR_W-1:0] addr_q;
    reg [DATA_W-1:0] wdata_q;
    reg [2:0]        wcnt;

    wire req      = read_enable | write_enable;
    wire coll     = read_enable & write_enable;
    wire range_bad = ({1'b0, address} >= addr_limit);
    wire wp_bad   = write_enable & wp;

    // ---------------- next-state logic ----------------
    always @(*) begin
        next_state = state;
        case (state)
            S_IDLE:
                if (en && req) begin
                    if (coll || range_bad || wp_bad) next_state = S_ERROR;
                    else if (write_enable)           next_state = S_WRITE;
                    else                             next_state = S_READ;
```

```
                end
            S_READ,
            S_WRITE: next_state = (wait_states != 3'd0) ? S_WAIT : S_DONE;
            S_WAIT : next_state = (wcnt == 3'd0) ? S_DONE : S_WAIT;
            S_DONE : next_state = S_IDLE;
            S_ERROR: next_state = S_IDLE;
            default: next_state = S_IDLE;
        endcase
    end

    // ---------------- state, request latch, wait counter ----------------
    always @(posedge clk) begin
        if (reset) begin
            state   <= S_IDLE;
            addr_q  <= {ADDR_W{1'b0}};
            wdata_q <= {DATA_W{1'b0}};
            wcnt    <= 3'd0;
        end else begin
            state <= next_state;
            if (state == S_IDLE && en && req) begin
                addr_q  <= address;
                wdata_q <= write_data;
            end
            if ((state == S_READ || state == S_WRITE) && wait_states != 3'd0)
                wcnt <= wait_states - 3'd1;
            else if (state == S_WAIT && wcnt != 3'd0)
                wcnt <= wcnt - 3'd1;
        end
    end

    // ---------------- error event pulses (registered) ----------------
    always @(posedge clk) begin
        if (reset) begin
            ev_coll <= 1'b0; ev_range <= 1'b0; ev_wpv <= 1'b0; ev_ovr <= 1'b0;
        end else begin
            ev_coll  <= (state == S_IDLE) && en && coll;
            ev_range <= (state == S_IDLE) && en && req && !coll && range_bad;
            ev_wpv   <= (state == S_IDLE) && en && !coll && !range_bad && wp_bad;
            ev_ovr   <= (state != S_IDLE) && req;
        end
    end

    // ---------------- Moore outputs ----------------
    assign busy      = (state != S_IDLE);
    assign ready     = (state == S_DONE) || (state == S_ERROR);
    assign error     = (state == S_ERROR);
    assign mem_we    = (state == S_WRITE);
    assign mem_re    = (state == S_READ);
    assign mem_addr  = addr_q;
    assign mem_wdata = wdata_q;
endmodule
```

7.3 cfg_regs.v

```
//=============================================================================
// Module      : cfg_regs
// Project     : MOSART Project 8 - Programmable Memory Controller
// Author      : Shaun Praneeth Raj
// Description : Configuration and status register bank. Programs the
//               controller behaviour at run time.
//
```

```
//   Addr  Name    Bits   Access  Reset  Meaning
//   0x0   CTRL    [0]    RW      1      EN  : controller enable
//                 [1]    RW      0      WP  : write protect (writes rejected)
//   0x1   WAIT    [2:0]  RW      0      number of wait states per access
//   0x2   LIMIT   [4:0]  RW      16     number of valid words (addr < LIMIT)
//   0x3   STATUS  [0]    W1C     0      COLL : simultaneous read+write request
//                 [1]    W1C     0      RANGE: address >= LIMIT
//                 [2]    W1C     0      WPV  : write attempted while WP = 1
//                 [3]    W1C     0      OVR  : request dropped while busy
//                 [4]    RO      -      BUSY : controller busy (live)
//                 [7:5]  RO      0      reserved
//============================================================================
`timescale 1ns / 1ps
module cfg_regs #(
    parameter ADDR_W = 4
)(
    input  wire          clk,
    input  wire          reset,       // synchronous, active high
    // configuration port
    input  wire          cfg_we,
    input  wire [1:0]    cfg_addr,
    input  wire [7:0]    cfg_wdata,
    output reg  [7:0]    cfg_rdata,
    // decoded controls to the FSM
    output reg           en,
    output reg           wp,
    output reg  [2:0]    wait_states,
    output reg  [ADDR_W:0] addr_limit,
    // error events from the FSM (one-cycle pulses)
    input  wire          ev_coll,
    input  wire          ev_range,
    input  wire          ev_wpv,
    input  wire          ev_ovr,
    input  wire          busy
);
    localparam A_CTRL = 2'd0, A_WAIT = 2'd1, A_LIMIT = 2'd2, A_STAT = 2'd3;

    reg [3:0] sticky;   // {OVR, WPV, RANGE, COLL}

    wire [3:0] clr = (cfg_we && cfg_addr == A_STAT) ? cfg_wdata[3:0] : 4'b0;
    wire [3:0] set = {ev_ovr, ev_wpv, ev_range, ev_coll};

    always @(posedge clk) begin
        if (reset) begin
            en          <= 1'b1;
            wp          <= 1'b0;
            wait_states <= 3'd0;
            addr_limit  <= (1 << ADDR_W);
            sticky      <= 4'b0;
        end else begin
            if (cfg_we) begin
                case (cfg_addr)
                    A_CTRL : begin en <= cfg_wdata[0]; wp <= cfg_wdata[1]; end
                    A_WAIT : wait_states <= cfg_wdata[2:0];
                    A_LIMIT: addr_limit  <= (cfg_wdata[ADDR_W:0] > (1 << ADDR_W))
                                          ? (1 << ADDR_W) : cfg_wdata[ADDR_W:0];
                    default: ;
                endcase
            end
            // a new event in the same cycle wins over the clear
            sticky <= (sticky & ~clr) | set;
        end
    end

```

```
72      // combinational read-back
73      always @(*) begin
74          case (cfg_addr)
75              A_CTRL : cfg_rdata = {6'b0, wp, en};
76              A_WAIT : cfg_rdata = {5'b0, wait_states};
77              A_LIMIT: cfg_rdata = {{(7-ADDR_W){1'b0}}, addr_limit};
78              default: cfg_rdata = {3'b0, busy, sticky};
79          endcase
80      end
81  endmodule
```

7.4 sync_mem.v

```
1   //============================================================================
2   // Module      : sync_mem
3   // Project     : MOSART Project 8 - Programmable Memory Controller
4   // Author      : Shaun Praneeth Raj
5   // Description : Single-port synchronous on-chip memory built from a register
6   //               array. Write is synchronous; read data is registered, so the
7   //               word addressed while mem_re is high appears on dout one clock
8   //               later and is held until the next read.
9   //============================================================================
10  `timescale 1ns / 1ps
11  module sync_mem #(
12      parameter DATA_W = 8,
13      parameter ADDR_W = 4
14  )(
15      input  wire              clk,
16      input  wire              mem_we,
17      input  wire              mem_re,
18      input  wire [ADDR_W-1:0] mem_addr,
19      input  wire [DATA_W-1:0] mem_wdata,
20      output reg  [DATA_W-1:0] mem_rdata
21  );
22      localparam DEPTH = (1 << ADDR_W);
23
24      reg [DATA_W-1:0] mem [0:DEPTH-1];
25
26      integer i;
27      initial begin
28          for (i = 0; i < DEPTH; i = i + 1)
29              mem[i] = {DATA_W{1'b0}};
30          mem_rdata = {DATA_W{1'b0}};
31      end
32
33      always @(posedge clk) begin
34          if (mem_we)
35              mem[mem_addr] <= mem_wdata;
36          if (mem_re)
37              mem_rdata <= mem[mem_addr];
38      end
39  endmodule
```

8 Testbench

The testbench `tb_mem_ctrl` instantiates `mem_ctrl_top` with a 100 MHz clock. Stimulus is applied on the falling edge and outputs are sampled on the falling edge after the rising edge that produced

them, which removes any race between testbench and design. A reference array `model[]` is updated only by accepted writes; every read is compared against it. Reusable tasks (`cfg_write`, `cfg_expect`, `do_access`, `mem_write`, `mem_read`) keep the test sequences short, and `do_access` also measures the latency of each transaction and checks that `busy` stays high until `ready`.

Table 6: Verification plan

Test	Scenario	Checks
T1	Reset values	CTRL = 0x01, WAIT = 0, LIMIT = 16, STATUS = 0, busy and ready low
T2	Write all 16 words, read all back, WAIT = 0	Data integrity at every address; latency = 2 cycles
T3	WAIT programmed to 1 through 7	Latency = 2 + WAIT for both reads and writes; busy held throughout
T4	`read_enable` and `write_enable` together	error pulse, STATUS.COLL set, memory unchanged, W1C clear
T5	LIMIT = 8, access 0xA and 0xC, then 0x7	RANGE error for out-of-window, normal access at 0x7, oversize LIMIT saturates to 16
T6	WP = 1 and a write	WPV error, memory unchanged, reads still allowed
T7	Second request while busy (WAIT = 4)	Request dropped, STATUS.OVR set, first write completes
T8	EN = 0 with request held for four cycles	busy and ready stay low, memory unchanged
T9	Reset during a read with WAIT = 5	FSM returns to IDLE, configuration to defaults, memory contents preserved
T10	500 random reads and writes, WAIT re-randomised every 50 accesses, then full read-back	Every read matches the reference model

```
//============================================================================
// Testbench   : tb_mem_ctrl
// Project     : MOSART Project 8 - Programmable Memory Controller
// Author      : Shaun Praneeth Raj
// Description : Self-checking testbench. A behavioural reference model
//               (model[]) mirrors every accepted write; every read and every
//               status/config read-back is compared against it. Stimulus is
//               driven on the falling edge, outputs are sampled on the
//               rising edge. Ends with a PASS/FAIL summary.
//============================================================================
`timescale 1ns/1ps
module tb_mem_ctrl;
    localparam DATA_W = 8, ADDR_W = 4, DEPTH = 16;
    localparam CLK_P  = 10;                         // 100 MHz

    reg              clk = 0, reset = 1;
    reg              read_enable = 0, write_enable = 0;
    reg  [ADDR_W-1:0] address = 0;
    reg  [DATA_W-1:0] write_data = 0;
    wire [DATA_W-1:0] read_data;
    wire             ready, busy, error;
    reg              cfg_we = 0;
    reg  [1:0]       cfg_addr = 0;
    reg  [7:0]       cfg_wdata = 0;
    wire [7:0]       cfg_rdata;

    mem_ctrl_top #(.DATA_W(DATA_W), .ADDR_W(ADDR_W)) dut (
        .clk(clk), .reset(reset),
        .read_enable(read_enable), .write_enable(write_enable),
```

```
        .address(address), .write_data(write_data), .read_data(read_data),
        .ready(ready), .busy(busy), .error(error),
        .cfg_we(cfg_we), .cfg_addr(cfg_addr),
        .cfg_wdata(cfg_wdata), .cfg_rdata(cfg_rdata)
    );

    always #(CLK_P/2) clk = ~clk;

    // ---------------- reference model & bookkeeping ----------------
    reg [DATA_W-1:0] model [0:DEPTH-1];
    integer errors = 0, checks = 0, test_errors_start = 0;
    integer lat, k, n, a, d, op, ws;
    integer n_rd = 0, n_wr = 0, n_rej = 0;
    reg [7:0] cur_wait = 0;
    reg [4:0] cur_limit = 16;
    reg       cur_wp = 0;
    reg       got_err;

    task check(input [639:0] what, input [31:0] got, input [31:0] exp);
        begin
            checks = checks + 1;
            if (got !== exp) begin
                errors = errors + 1;
                $display("  ** FAIL %0s : got 0x%0h expected 0x%0h @ %0t",
                         what, got, exp, $time);
            end
        end
    endtask

    task begin_test(input [639:0] name);
        begin
            test_errors_start = errors;
            $display("[%0t] TEST: %0s", $time, name);
        end
    endtask

    task end_test(input [639:0] name);
        begin
            if (errors == test_errors_start) $display("    -> PASS  %0s", name);
            else                             $display("    -> FAIL  %0s", name);
        end
    endtask

    // ---------------- config bus ----------------
    task cfg_write(input [1:0] ad, input [7:0] dat);
        begin
            @(negedge clk); cfg_we = 1; cfg_addr = ad; cfg_wdata = dat;
            @(negedge clk); cfg_we = 0;
        end
    endtask

    task cfg_expect(input [1:0] ad, input [7:0] exp);
        begin
            @(negedge clk); cfg_addr = ad; #1;
            check("cfg readback", cfg_rdata, exp);
        end
    endtask

    // ---------------- host transactions ----------------
    // Issues one request, waits for ready, returns latency (cycles from the
    // edge that samples the request to the edge that samples ready) and
    // whether the controller flagged an error.
    task do_access(input rd, input wr, input [ADDR_W-1:0] ad,
                   input [DATA_W-1:0] dat, output integer cycles,
```

```
                output reg err);
        begin
            @(negedge clk);
            read_enable = rd; write_enable = wr; address = ad; write_data = dat;
            @(negedge clk);
            read_enable = 0; write_enable = 0;
            cycles = 1;
            while (!ready) begin
                check("busy during access", busy, 1);
                @(negedge clk); cycles = cycles + 1;
            end
            err = error;
        end
    endtask

    task mem_write(input [ADDR_W-1:0] ad, input [DATA_W-1:0] dat,
                input exp_err);
        begin
            do_access(0, 1, ad, dat, lat, got_err);
            check("write error flag", got_err, exp_err);
            if (!got_err) begin model[ad] = dat; n_wr = n_wr + 1; end
            else n_rej = n_rej + 1;
        end
    endtask

    task mem_read(input [ADDR_W-1:0] ad, input exp_err);
        begin
            do_access(1, 0, ad, 8'h00, lat, got_err);
            check("read error flag", got_err, exp_err);
            if (!got_err) begin
                check("read data", read_data, model[ad]);
                n_rd = n_rd + 1;
            end else n_rej = n_rej + 1;
        end
    endtask

    // ---------------- stimulus ----------------
    initial begin
        $dumpfile("mem_ctrl.vcd");
        $dumpvars(0, tb_mem_ctrl);
        for (k = 0; k < DEPTH; k = k + 1) model[k] = 0;

        repeat (3) @(negedge clk);
        reset = 0;

        //-------------------------------------------------------------
        begin_test("T1 reset values of configuration registers");
        cfg_expect(2'd0, 8'h01);   // EN=1, WP=0
        cfg_expect(2'd1, 8'h00);   // WAIT=0
        cfg_expect(2'd2, 8'h10);   // LIMIT=16
        cfg_expect(2'd3, 8'h00);   // STATUS clear, not busy
        check("busy after reset", busy, 0);
        check("ready after reset", ready, 0);
        end_test("T1");

        //-------------------------------------------------------------
        begin_test("T2 write all 16 words, read back, zero wait states");
        for (k = 0; k < DEPTH; k = k + 1) begin
            mem_write(k, 8'hA0 ^ (k * 8'h11), 0);
            check("write latency WAIT=0", lat, 2);
        end
        for (k = 0; k < DEPTH; k = k + 1) begin
            mem_read(k, 0);
            check("read latency WAIT=0", lat, 2);
```

```
158         end
159         end_test("T2");
160
161         //------------------------------------------------------------
162         begin_test("T3 programmable wait states 1..7");
163         for (ws = 1; ws < 8; ws = ws + 1) begin
164             cfg_write(2'd1, ws);
165             mem_write(ws, 8'h30 + ws, 0);
166             check("write latency", lat, 2 + ws);
167             mem_read(ws, 0);
168             check("read latency", lat, 2 + ws);
169         end
170         cfg_write(2'd1, 8'd3);
171         mem_write(4'hF, 8'h5A, 0);
172         mem_read (4'hF, 0);
173         check("read latency WAIT=3", lat, 5);
174         cfg_write(2'd1, 8'd0);
175         end_test("T3");
176
177         //------------------------------------------------------------
178         begin_test("T4 simultaneous read + write request is rejected");
179         do_access(1, 1, 4'h2, 8'hEE, lat, got_err);
180         check("collision error", got_err, 1);
181         n_rej = n_rej + 1;
182         cfg_expect(2'd3, 8'h01);   // COLL set
183         mem_read(4'h2, 0);         // memory must be unchanged
184         cfg_write(2'd3, 8'h01);    // W1C
185         cfg_expect(2'd3, 8'h00);
186         end_test("T4");
187
188         //------------------------------------------------------------
189         begin_test("T5 address limit (LIMIT = 8)");
190         cfg_write(2'd2, 8'd8);
191         cfg_expect(2'd2, 8'd8);
192         mem_write(4'd10, 8'h99, 1);    // out of range -> error
193         mem_read (4'd12, 1);
194         cfg_expect(2'd3, 8'h02);       // RANGE set
195         mem_write(4'd7, 8'h77, 0);     // last valid word
196         mem_read (4'd7, 0);
197         cfg_write(2'd2, 8'd31);        // oversize value saturates to 16
198         cfg_expect(2'd2, 8'd16);
199         mem_read (4'd10, 0);           // old contents intact
200         cfg_write(2'd3, 8'h0F);
201         end_test("T5");
202
203         //------------------------------------------------------------
204         begin_test("T6 write protect");
205         cfg_write(2'd0, 8'h03);        // EN=1, WP=1
206         mem_write(4'd3, 8'hFF, 1);
207         cfg_expect(2'd3, 8'h04);       // WPV set
208         mem_read (4'd3, 0);            // reads still allowed, data unchanged
209         cfg_write(2'd0, 8'h01);
210         cfg_write(2'd3, 8'h0F);
211         mem_write(4'd3, 8'h3C, 0);
212         mem_read (4'd3, 0);
213         end_test("T6");
214
215         //------------------------------------------------------------
216         begin_test("T7 request while busy is dropped and flagged");
217         cfg_write(2'd1, 8'd4);
218         @(negedge clk);
219         write_enable = 1; address = 4'd5; write_data = 8'h55;
220         @(negedge clk);
221         write_enable = 1; address = 4'd6; write_data = 8'h66;  // arrives while busy
```

```
222         @(negedge clk);
223         write_enable = 0;
224         while (!ready) @(negedge clk);
225         model[5] = 8'h55; n_wr = n_wr + 1;
226         cfg_expect(2'd3, 8'h08);       // OVR set
227         mem_read(4'd6, 0);             // location 6 must NOT be 0x66
228         mem_read(4'd5, 0);
229         cfg_write(2'd3, 8'h0F);
230         cfg_write(2'd1, 8'd0);
231         end_test("T7");
232
233         //-------------------------------------------------------------
234         begin_test("T8 controller disabled (EN = 0) ignores requests");
235         cfg_write(2'd0, 8'h00);
236         @(negedge clk);
237         write_enable = 1; address = 4'd1; write_data = 8'hBD;
238         repeat (4) begin
239             @(negedge clk);
240             check("busy while disabled", busy, 0);
241             check("ready while disabled", ready, 0);
242         end
243         write_enable = 0;
244         cfg_write(2'd0, 8'h01);
245         mem_read(4'd1, 0);             // not overwritten
246         end_test("T8");
247
248         //-------------------------------------------------------------
249         begin_test("T9 reset during a transaction");
250         cfg_write(2'd1, 8'd5);
251         @(negedge clk);
252         read_enable = 1; address = 4'd9;
253         @(negedge clk); read_enable = 0;
254         @(negedge clk); @(negedge clk);
255         check("busy mid-transaction", busy, 1);
256         reset = 1; @(negedge clk); reset = 0;
257         check("busy after reset", busy, 0);
258         cfg_expect(2'd1, 8'd0);        // config back to default
259         mem_read(4'd9, 0);             // memory array keeps its contents
260         end_test("T9");
261
262         //-------------------------------------------------------------
263         begin_test("T10 constrained-random regression (500 accesses)");
264         n = 0;
265         for (k = 0; k < 500; k = k + 1) begin
266             if (k % 50 == 0) begin
267                 cfg_write(2'd1, $urandom % 8);
268             end
269             a  = $urandom % DEPTH;
270             d  = $urandom % 256;
271             op = $urandom % 2;
272             if (op) mem_write(a, d, 0);
273             else    mem_read (a, 0);
274         end
275         for (k = 0; k < DEPTH; k = k + 1) mem_read(k, 0);
276         end_test("T10");
277
278         //-------------------------------------------------------------
279         repeat (3) @(negedge clk);
280         $display("============================================================");
281         $display(" SUMMARY: %0d checks, %0d errors | %0d writes, %0d reads, %0d rejected",
282                  checks, errors, n_wr, n_rd, n_rej);
283         if (errors == 0) $display(" RESULT : ALL TESTS PASSED");
284         else             $display(" RESULT : FAILURES DETECTED");
285         $display("============================================================");
```

```
286         $finish;
287     end
288
289     // watchdog
290     initial begin
291         #2_000_000;
292         $display("TIMEOUT"); $finish;
293     end
294 endmodule
```

9 Simulation Results

9.1 Simulation Log

The testbench was compiled and run with Icarus Verilog 12.0 (`iverilog -g2005`). The same sources can be run in the Vivado simulator with `vivado/run_sim.tcl`. The console output is reproduced below.

```
[30 ns]   TEST: T1 reset values of configuration registers        -> PASS
[71 ns]   TEST: T2 write all 16 words, read back, zero wait states -> PASS
[1030 ns] TEST: T3 programmable wait states 1..7                   -> PASS
[2310 ns] TEST: T4 simultaneous read + write request is rejected   -> PASS
[2401 ns] TEST: T5 address limit (LIMIT = 8)                       -> PASS
[2620 ns] TEST: T6 write protect                                   -> PASS
[2800 ns] TEST: T7 request while busy is dropped and flagged       -> PASS
[3080 ns] TEST: T8 controller disabled (EN = 0) ignores requests   -> PASS
[3200 ns] TEST: T9 reset during a transaction                      -> PASS
[3310 ns] TEST: T10 constrained-random regression (500 accesses)   -> PASS
============================================================
 SUMMARY: 3356 checks, 0 errors | 281 writes, 295 reads, 4 rejected
 RESULT : ALL TESTS PASSED
============================================================
$finish called at 36680 ns
```

To confirm that the checker is capable of detecting faults, two deliberate bugs were injected into a copy of the FSM. Removing the collision check made T4 fail (2 errors), and loading the wait counter with `WAIT` instead of `WAIT` $-$ 1 made T3 fail (15 latency errors). The original RTL passes every check.

9.2 Simulation Waveforms

All waveforms below are drawn in TikZ directly from the VCD dump of the passing run. Values on buses are hexadecimal unless stated otherwise; vertical grid lines mark rising clock edges.

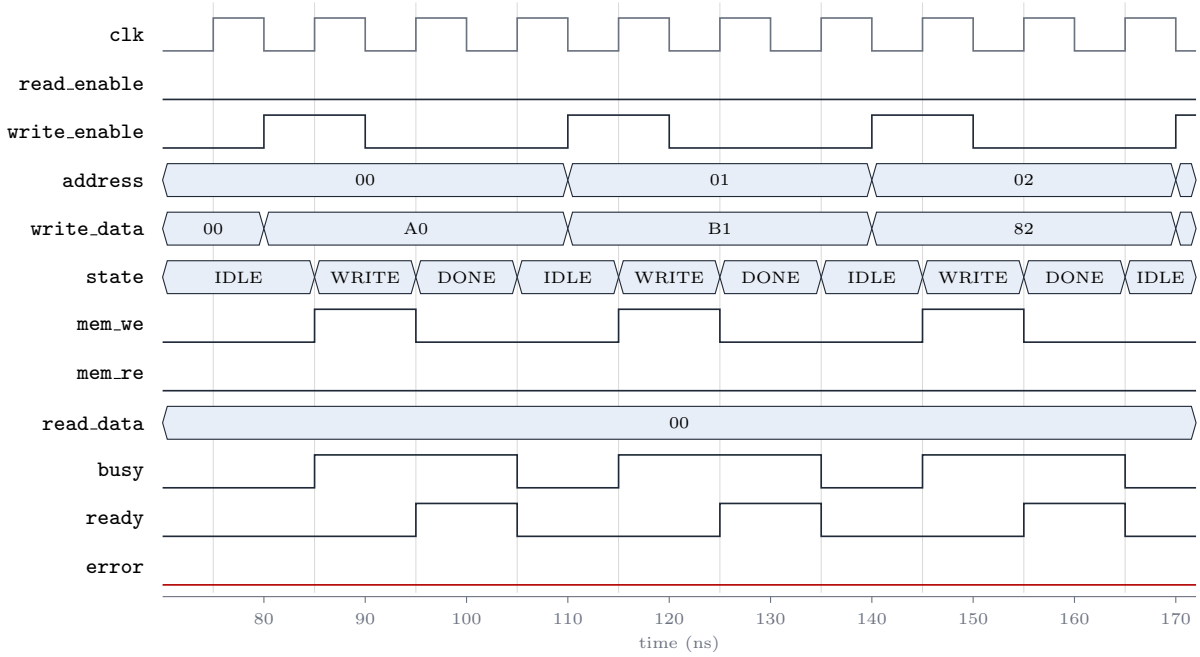


Figure 5: Back-to-back writes with zero wait states

Each write is sampled in IDLE, spends one cycle in WRITE with `mem_we` high, then one cycle in DONE with `ready` high. `busy` rises on the cycle after the request and falls when the FSM returns to IDLE; a new write follows every three cycles.

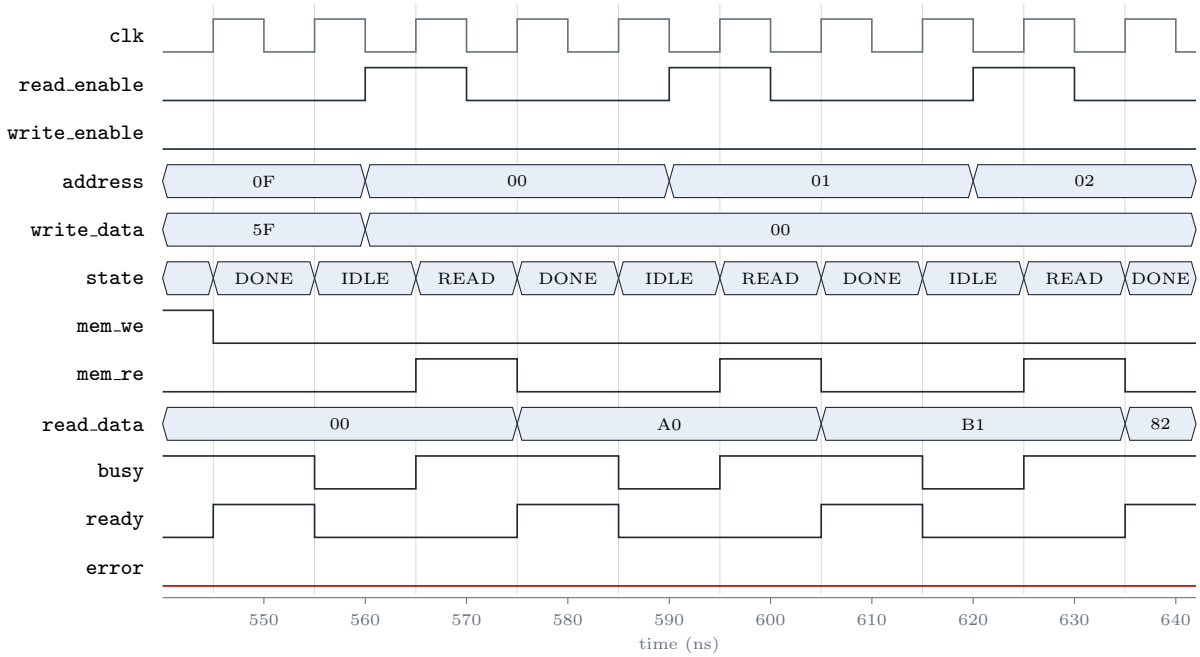


Figure 6: Back-to-back reads with zero wait states

`mem_re` is high during READ; the registered memory output updates on the edge that ends READ, so `read_data` already holds the addressed word when `ready` is high in DONE, and it stays stable afterwards.

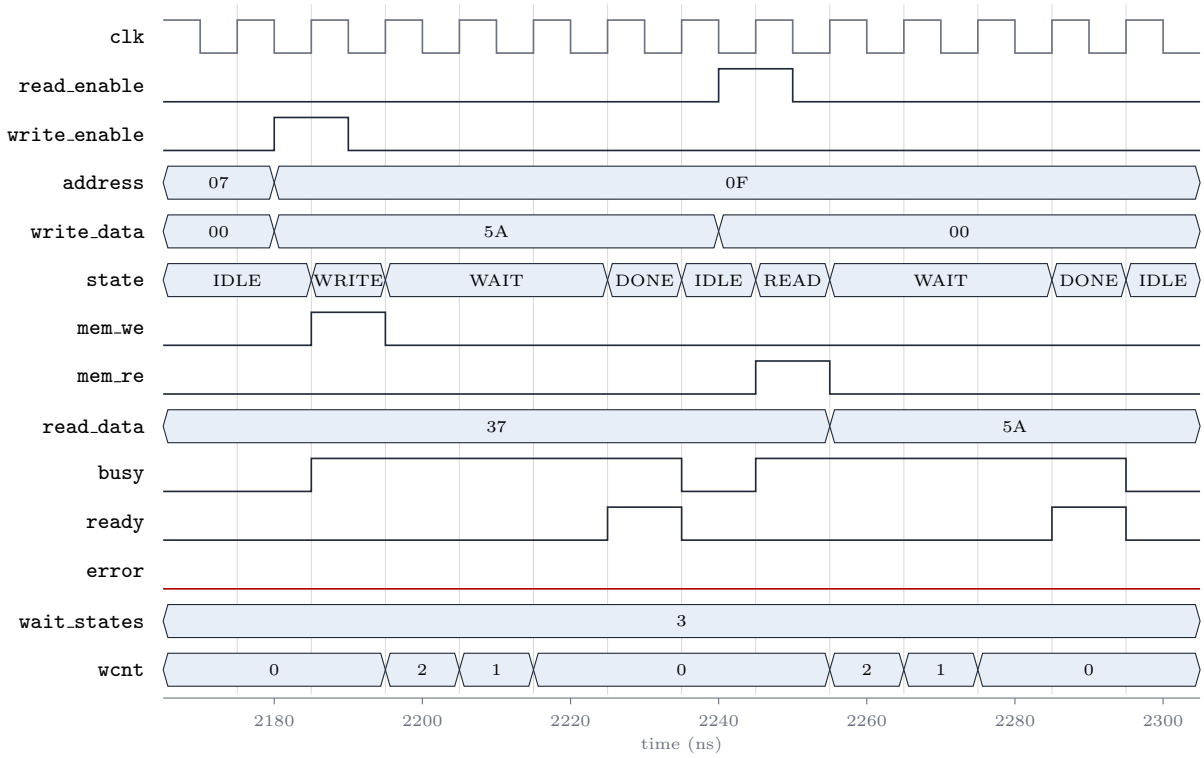


Figure 7: Write and read of address 0xF with three programmed wait states

With WAIT = 3 the FSM passes through WAIT for exactly three cycles while `wcnt` counts 2, 1, 0. Latency grows from 2 to 5 cycles and `busy` remains high for the whole transaction. The read returns 0x5A, the value written just before.

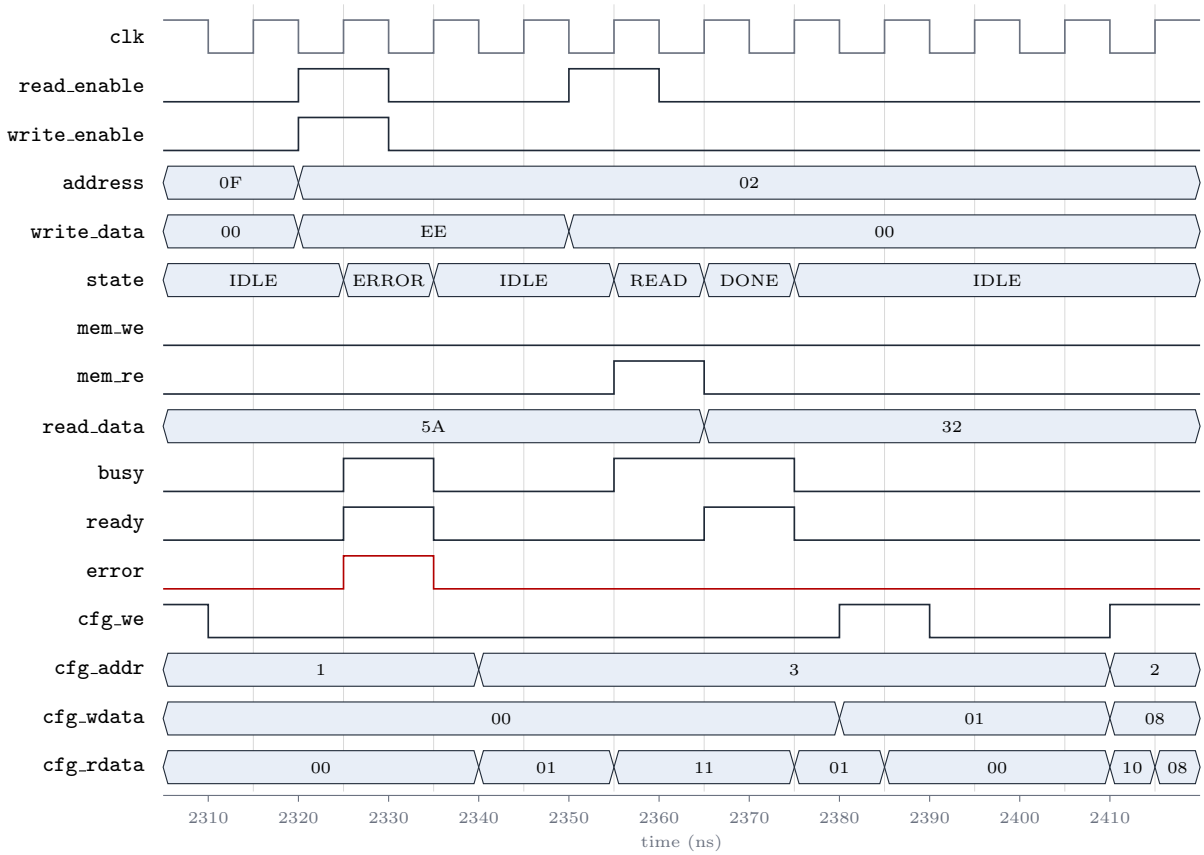


Figure 8: Simultaneous read and write request rejected

Both enables are high in the same cycle, so the FSM goes to ERROR instead of READ or WRITE. `error` and `ready` pulse together, neither `mem_we` nor `mem_re` is asserted, and a STATUS read-back returns 0x01 (COLL). The following read of address 0x2 confirms that memory is unchanged, and writing 0x01 to STATUS clears the bit.

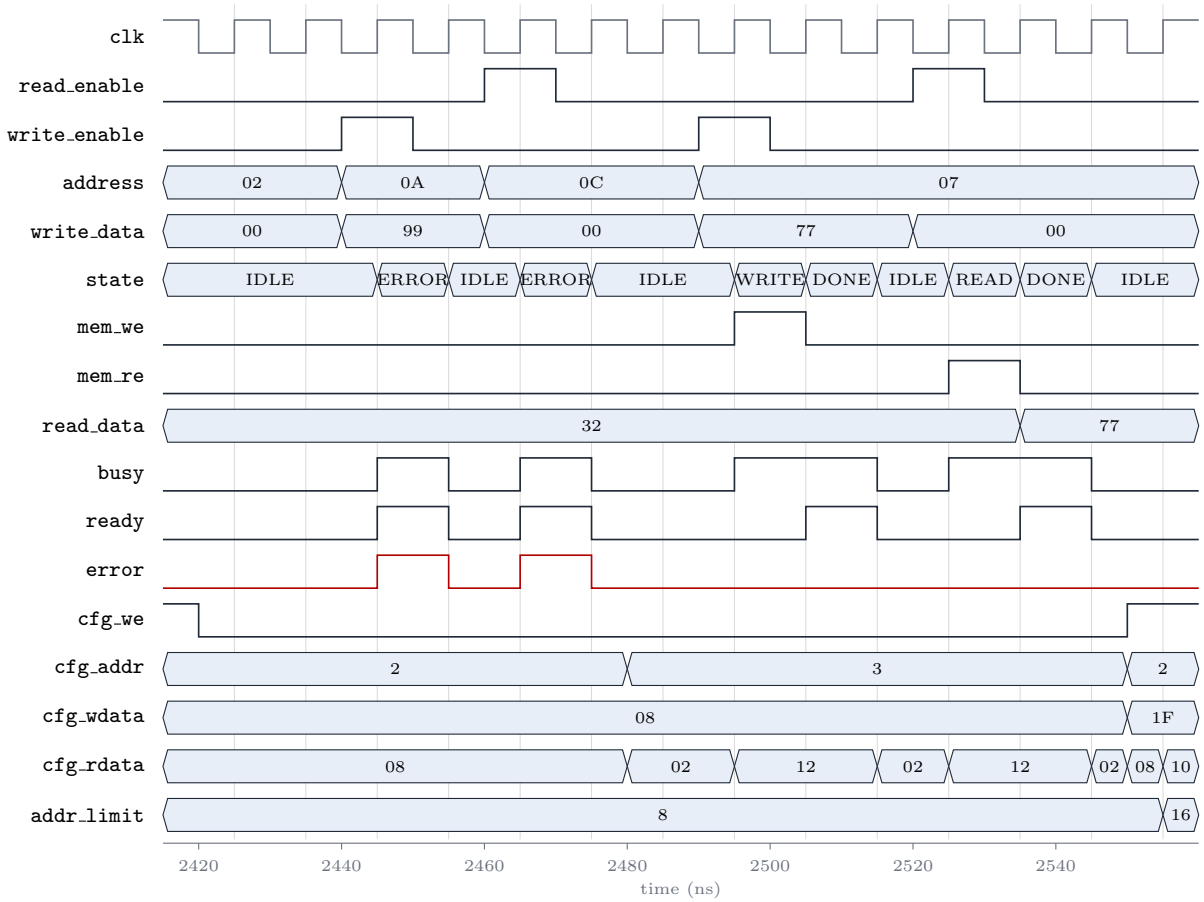


Figure 9: Address window limited to eight words

With LIMIT = 8, the write to 0xA and the read of 0xC are rejected through ERROR. STATUS reads 0x02 (RANGE) when idle and 0x12 while a later access is in progress, because bit 4 mirrors `busy`. The write and read at 0x7, the last valid word, complete normally and return 0x77. Writing 0x1F to LIMIT saturates the register to 16.

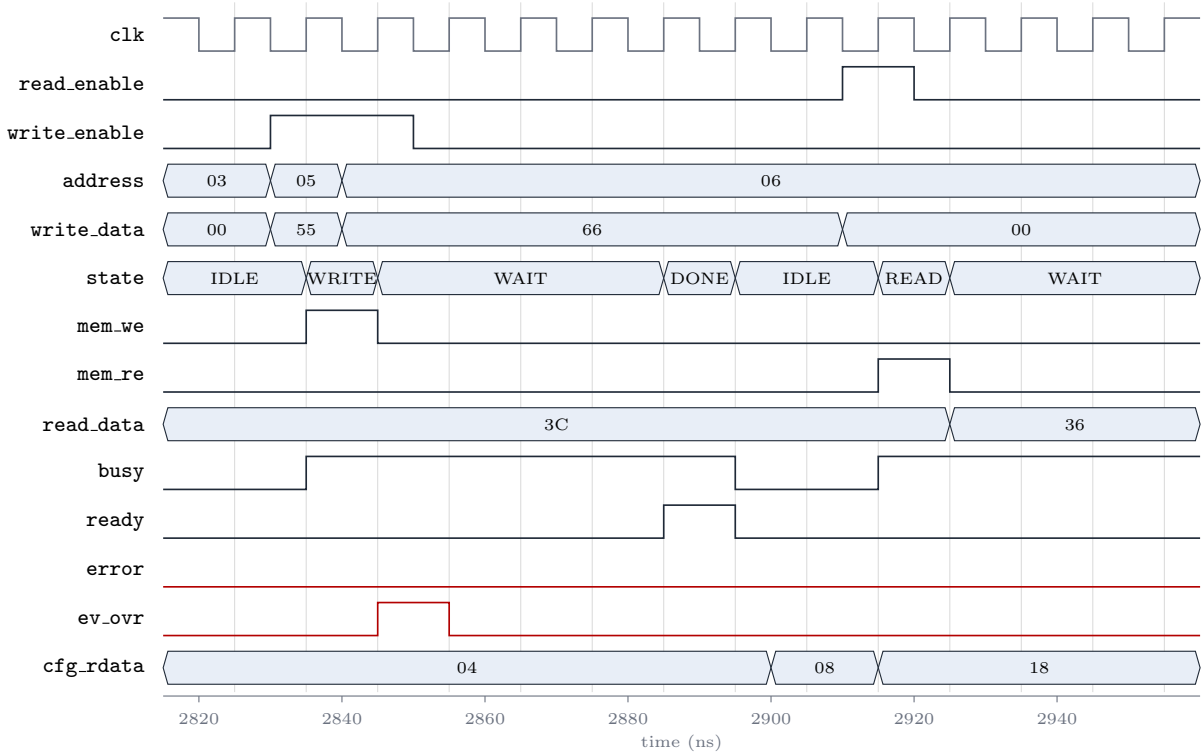


Figure 10: Request arriving while the controller is busy

A write to 0x5 is accepted; one cycle later a write to 0x6 arrives while `busy` is high. It is not latched, `ev_ovr` pulses, and STATUS reads 0x08 (OVR) once the controller is idle. The subsequent reads show that 0x5 holds 0x55 and 0x6 was not written.

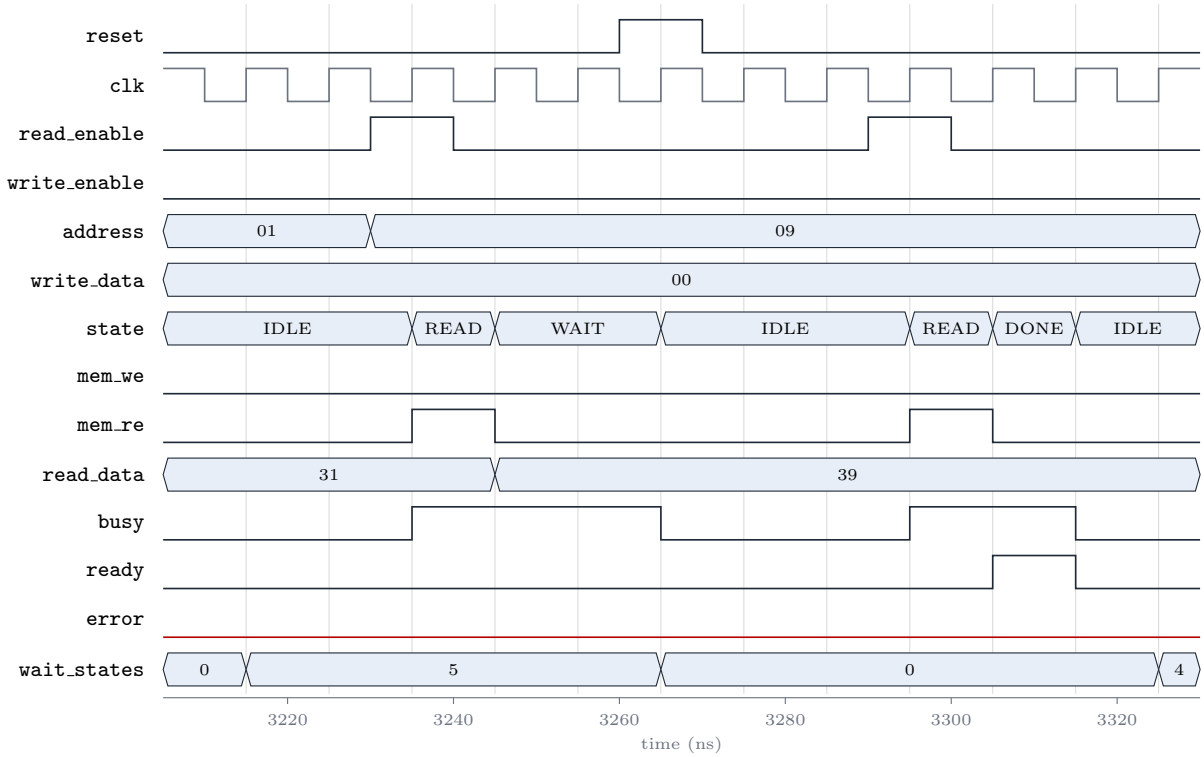


Figure 11: Synchronous reset applied in the middle of a read

Reset is asserted while the FSM is in WAIT during a five-wait-state read. On the next edge the FSM returns to IDLE, `busy` falls and WAIT reverts to 0. A read of the same address afterwards returns the stored value, showing that the memory array survives a controller reset.

9.3 Important Signal Explanations

Table 7: Signals shown in the waveforms

Signal	Meaning in the waveforms
`state`	FSM state register, decoded to state names
`mem_we` / `mem_re`	Internal memory strobes; exactly one cycle wide per accepted access, never asserted for a rejected one
`busy`	High from the cycle after a request is accepted until the FSM re-enters IDLE
`ready`	One-cycle completion pulse in DONE or ERROR
`error`	One-cycle rejection pulse, always together with `ready`
`wcnt`	Wait-state down-counter, loaded with WAIT − 1 when leaving READ or WRITE
`ev_ovr`	Internal pulse that sets STATUS.OVR when a request arrives while busy
`cfg_rdata`	Combinational read-back of the register selected by `cfg_addr`
`addr_limit`	Current LIMIT register value (decimal)

9.4 Verification Summary

Table 8: Evidence that every specified function has been verified

Requirement	Tests	Evidence
R1 Read requests	T2, T3, T10	295 reads checked against model; Fig. 6
R2 Write requests	T2, T3, T10	281 accepted writes; Fig. 5
R3 Memory control signals	T2 to T7	One-cycle strobes; none on rejection; Figs. 5, 8
R4 Read data	T2, T3, T10	Data valid with `ready`; Fig. 7
R5 BUSY / READY	All	busy checked every cycle of every access
R6 Simultaneous requests	T4	Fig. 8
R7 Invalid requests	T5, T6, T7	Figs. 9, 10
R8 Programmability	T1, T3, T5, T6, T8	Latency 2 + WAIT for WAIT = 0 to 7; LIMIT, WP and EN behaviour
R9 Register-array memory	T2, T9, T10	All 16 words; contents kept across reset; Fig. 11

10 Synthesis Results

The target is the Artix-7 device xc7a35tcpg236-1 (Basys-3 board) with a 100 MHz clock constraint (`vivado/mem_ctrl.xdc`). The Vivado flow is scripted in `vivado/run_synth.tcl`, which runs `synth_design` and writes the utilization, hierarchical utilization, timing summary and critical path reports.

The figures in this section come from Yosys 0.33 with its Xilinx 7-series flow (`synth_xilinx -family xc7`), which maps to the same primitive library (LUT1 to LUT6, MUXF7/F8, CARRY4, FDRE/FDSE, RAM32M) that Vivado uses. Synthesis completed with no latches inferred, and the final `check` pass reported no problems (no combinational loops, no undriven nets). The cell report is reproduced in Table 9.

Table 9: Yosys `synth_xilinx` cell report for `mem_ctrl_top`

Primitive	Count	Used for
LUT2	21	
LUT3	11	
LUT4	4	Next-state logic, validation, decode, read-back mux
LUT5	4	
LUT6	19	
MUXF7 / MUXF8	7 / 3	Wide multiplexers in next-state and read-back logic
CARRY4	1	Wait-counter decrement
RAM32M	1	16×8 memory array as distributed RAM
FDRE	42	Registers with synchronous reset to 0
FDSE	2	Registers with synchronous set (EN, LIMIT[4])
IBUF / OBUF / BUFG	27 / 19 / 1	I/O buffers and global clock buffer

11 Timing Analysis

The timing target is a 10 ns period (100 MHz). Because the design is a single synchronous domain with a synchronous reset, the only paths of interest are register-to-register paths inside `clk`, plus input-to-register paths from the host pins.

A longest-path analysis of the mapped netlist (counting LUT, MUXF and CARRY cells between sequential elements) gives the following:

Table 10: Deepest logic paths in the mapped netlist

Levels	Endpoint	Path
6	`state[1:0]`	`address` / LIMIT $\rightarrow \geq$ compare $\rightarrow$ IDLE decision (LUT6, MUXF7, MUXF8, LUT6, MUXF7, MUXF8)
5	`state[2]`	same sources, one level shorter
4	`ev_range`, `ev_wpv`	same comparator feeding the error-event registers
3	`wcnt[2]`	state and WAIT $\rightarrow$ CARRY4 decrement

The critical path is therefore the IDLE request decision: the 5-bit address-versus-LIMIT comparison combined with the collision and write-protect checks selects among four next states. A MUXF7/MUXF8 pair adds very little delay on 7-series parts, so six levels of this kind are comfortably within a 10 ns budget; a typical Artix-7 -1 LUT plus local routing costs well under 1 ns per level. The exact worst negative slack (WNS), total negative slack and achievable $f_{max} = 1/(T - \text{WNS})$ are read from `reports/timing_summary.rpt` produced by `run_synth.tcl`.

Throughput and latency. At 100 MHz with WAIT = 0, an access completes 20 ns after the request is sampled and a new access can start every 30 ns, giving 33.3 million accesses per second. Each wait state adds 10 ns to both figures.

12 Resource Utilization

Table 11: Resource utilization on xc7a35tcpg236-1 (from the Yosys 7-series mapping)

Resource	Used	Available	Utilization
Slice LUTs as logic	59	20800	0.28%
Slice LUTs as distributed RAM (RAM32M)	4	9600	0.04%
Slice registers (flip-flops)	44	41600	0.11%
F7 / F8 muxes	7 / 3	16300 / 8150	<0.05%
Bonded IOB	46	106	43.4%
BUFGCTRL	1	32	3.1%

The 44 flip-flops account exactly for the registers in the RTL:

Table 12: Flip-flop budget by function

Block	Registers	Bits
FSM	state (3), wcnt (3), addr_q (4), wdata_q (8), error events (4)	22
Configuration	EN (1), WP (1), WAIT (3), LIMIT (5), STATUS sticky bits (4)	14
Memory	registered read output `mem_rdata`	8
Total		**44**

The 128 storage bits of the array do not appear as flip-flops: because the array has no reset and a single synchronous write port, it is inferred as one RAM32M distributed-RAM primitive (four LUTs used as memory). Vivado normally performs the same inference; if it instead builds the array from registers, the flip-flop count rises by 128 and the LUT count by the read multiplexer, still well below 1% of the device. The IOB figure is high only because every port is brought to a pin for a standalone build; when the controller is embedded in a larger design these ports become internal nets.

13 Design Challenges

1. **Read-data alignment.** The memory has a registered output, so the read value appears one cycle after `mem_re`. Placing DONE directly after READ (or after WAIT) and driving `read_data` straight from the memory register makes the data valid exactly when `ready` is high, without an extra holding register.

2. **Wait-state counting off-by-one.** Loading the counter with WAIT and leaving WAIT on zero produced one extra cycle. The counter is loaded with WAIT − 1 when leaving READ or WRITE; the latency check in T3 (2 + WAIT for all eight settings) caught the incorrect variant during mutation testing.

3. **Defining "simultaneous" and "invalid".** The specification leaves the policy open. Rejecting a collision, rather than giving priority to one operation, was chosen because silently dropping one of two requests hides a bug in the requesting block. A fixed check order (collision, range, protection) gives each rejected request exactly one reason in STATUS.

4. **Sticky status versus clear.** An error event arriving in the same cycle as a W1C clear could be lost. The update `sticky <= (sticky & ~clr) | set` gives the event priority.

5. **Testbench races.** Driving and sampling on the same clock edge produced order-dependent results in early runs. Driving on the falling edge and sampling half a cycle after the active edge removed the ambiguity.

14 Possible Improvements

- **Pipelined back-to-back access.** Accepting a new request in DONE (or using a one-entry request queue) would raise throughput from one access per 3 cycles to one per 2 at WAIT = 0.
- **Shorter critical path.** Registering the address-range check (precomputing a valid flag when the address arrives) would remove the comparator from the next-state path.
- **Burst transfers.** A burst-length register and an auto-incrementing address counter would let one request move several consecutive words.
- **Standard bus interface.** Wrapping the request port as an AXI4-Lite or APB slave would allow direct connection to a MicroBlaze or Zynq processing system.
- **Block RAM and error protection.** For larger depths the array can be mapped to BRAM, and a Hamming SEC-DED code can be added to detect and correct single-bit upsets.
- **Assertion-based checks.** SystemVerilog assertions (for example, `mem_we` and `mem_re` never high together, `ready` always one cycle wide) and functional coverage would strengthen verification.

15 Conclusion

A programmable synchronous memory controller for a 16×8 on-chip register-array memory was designed in synthesizable Verilog, verified with a self-checking testbench that passed all 3356 checks across ten test groups, and synthesized for the Artix-7 fabric using 59 LUTs, one distributed RAM primitive and 44 flip-flops with a six-level critical path well within the 100 MHz target.